6주차 과제

(파머완 5장 개념 정리 및 필사; 5.9장, 5.10장 제외)

5장 회귀

5.1 회귀 소개

회귀의 개념

- **회귀(regression)**: 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법

  cf. 유래: 영국 통계학자 갈톤(Galton)의 부모-자식 키 연구 → 사람의 키는 **평균 키로 회귀**하려는 경향
- 통계학적 정의: 여러 개의 **독립변수**와 한 개의 **종속변수** 간의 상관관계를 모델링하는 기법

  ex. 아파트 방 개수, 방 크기, 주변 학군(독립변수) → 아파트 가격(종속변수)

$$Y = W_1X_1 + W_2X_2 + W_3X_3 + \cdots + W_nX_n$$

- Y: 종속변수 / X_1~X_n: 독립변수 / W_1~W_n: **회귀 계수(Regression coefficients)**
- 머신러닝 관점: 독립변수 = 피처, 종속변수 = 결정 값
- ⇒ 머신러닝 회귀 예측의 핵심 = 주어진 피처와 결정 값 데이터 기반에서 학습을 통해 **최적의 회귀 계수**를 찾아내는 것

회귀 유형 구분

독립변수 개수	회귀 계수의 결합
1개: 단일 회귀	선형: 선형 회귀
여러 개: 다중 회귀	비선형: 비선형 회귀

- 지도학습 = 분류 + 회귀
  - **분류**: 예측값이 카테고리와 같은 **이산형** 클래스 값
  - **회귀**: 예측값이 **연속형** 숫자 값

대표적인 선형 회귀 모델

💡 **선형 회귀**
:실제 값과 예측값의 차이(오류의 제곱 값)를 최소화하는 직선형 회귀선을 최적화하는 방식

규제(Regularization)
: 일반적인 선형 회귀의 과적합 문제를 해결하기 위해 회귀 계수에 패널티 값을 적용하는 것

- **일반 선형 회귀**: 예측값과 실제 값의 RSS를 최소화할 수 있도록 회귀 계수를 최적화, 규제 적용 X

- **릿지(Ridge)**: 선형 회귀 + **L2 규제** → 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해 회귀 계수값을 더 작게 만듦
- **라쏘(Lasso)**: 선형 회귀 + **L1 규제** → 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 함 (= 피처 선택 기능)
- **엘라스틱넷(ElasticNet)**: L2 + L1 규제 결합. 피처가 많은 데이터 세트에서 적용 → L1으로 피처 개수 줄임 + L2로 계수 값 크기 조정
- **로지스틱 회귀(Logistic Regression)**: 이름은 회귀지만 사실은 **분류**에 사용되는 선형 모델. 이진 분류뿐 아니라 희소 영역의 분류(ex. 텍스트 분류)에서 뛰어난 성능

5.2 단순 선형 회귀를 통한 회귀 이해

단순 선형 회귀

- 독립변수도 하나, 종속변수도 하나인 선형 회귀
  - ex. 주택 가격이 주택 크기로만 결정된다고 가정 → 주택 가격은 주택 크기에 대해 선형(직선 형태)의 관계로 표현 가능

$$\hat{Y} = w_0 + w_1 \cdot X$$

- w_1: 기울기 / w_0: 절편(intercept) → 단순 선형 회귀에서는 이 둘을 **회귀 계수**로 지칭
- 실제 주택 가격 = w_0 + w_1·X + 오류 값

잔차와 RSS

- **잔차**: 실제 값과 회귀 모델의 차이에 따른 오류 값(남은 오류)
- 최적의 회귀 모델 = 전체 데이터의 **잔차(오류 값) 합이 최소**가 되는 모델 = 오류 값 합이 최소가 되는 최적의 회귀 계수를 찾는 것
- 오류 값은 +나 -가 될 수 있음 → 단순히 더하면 오류 합이 크게 줄어들 수 있음
  - **MAE(Mean Absolute Error)**: 오류 값의 절댓값을 취해서 더함
  - **RSS(Residual Sum of Square)**: 오류 값의 제곱을 구해서 더함 ← 미분 등의 계산을 편리하게 하기 위해 일반적으로 사용 ($Error^2 = RSS$)

$$RSS(w_0, w_1) = (1/N) \cdot \Sigma_{i=1}^{N} (y_i - (w_0 + w_1 \cdot x_i))^2$$

- (i는 1부터 학습 데이터의 총 건수 N까지)
- RSS는 독립변수 X, 종속변수 Y가 아니라 **w 변수(회귀 계수)가 중심 변수**임에 유의 (학습 데이터로 입력되는 X, Y는 RSS에서 모두 상수로 간주)
- 회귀에서 RSS = 비용(Cost) → w 변수로 구성되는 RSS를 **비용 함수** 또는 **손실 함수(loss function)** 라고 함
- ⇒ 머신러닝 회귀 알고리즘은 데이터를 계속 학습하면서 비용 함수가 반환하는 값(오류 값)을 지속해서 감소시키고, 최종적으로 더 이상 감소하지 않는 최소의 오류 값을 구하는 것

5.3 비용 최소화하기 - 경사 하강법(Gradient Descent) 소개

경사 하강법의 개념

- W 파라미터가 많으면 고차원 방정식으로 비용 함수 최소 W를 구하기 어려움 → 경사 하강법이 이를 직관적으로 해결
- '데이터를 기반으로 알고리즘이 스스로 학습한다'는 머신러닝 개념을 가능하게 만든 핵심 기법
- **'점진적으로'** 반복적인 계산을 통해 W 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터를 구하는 방식
  - cf. 비유: 깜깜한 밤 산 정상에서 무조건 현재보다 낮은 곳으로 계속 이동하다 보면 지상에 도착
  - 단, 더 이상 내려갈 곳이 없는 가장 낮은 곳이라 여겼지만 오르막·내리막이 반복되는 언덕에 갇힐 수도 있음 (= local minimum)
- 핵심: **"어떻게 하면 오류가 작아지는 방향으로 W 값을 보정할 수 있을까?"**
  - 포물선 형태의 2차 함수의 최저점 = 미분 값인 1차 함수의 기울기가 가장 최소일 때
  - 최초 w에서부터 미분을 적용한 뒤 이 미분 값이 계속 감소하는 방향으로 순차적으로 w를 업데이트 → 더 이상 기울기가 감소하지 않는 지점을 비용 함수가 최소인 지점으로 간주하고 그때의 w 반환

경사 하강법 수식 정리

- 비용 함수

$$R(w) = (1/N) \cdot \Sigma_{i=1}^{N}(y_i - (w_0 + w_1 x_i))^2$$

R(w)는 두 개의 w 파라미터(w_0, w_1)를 가지므로 각 변수에 **편미분** 적용

$$\partial R(w)/\partial w_1 = -(2/N) \cdot \Sigma_{i=1}^{N} x_i \cdot (\text{실제값}_i - \text{예측값}_i)$$

$$\partial R(w)/\partial w_0 = -(2/N) \cdot \Sigma_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$$

- 업데이트: 새로운 w = 이전 w - 편미분 결괏값
- 편미분 값이 너무 클 수 있기 때문에 보정 계수 **η(학습률)** 를 곱함

$$\text{새로운} w_1 = \text{이전} w_1 + \eta \cdot (2/N) \cdot \Sigma_{i=1}^{N} x_i \cdot (\text{실제값}_i - \text{예측값}_i)$$

$$\text{새로운} w_0 = \text{이전} w_0 + \eta \cdot (2/N) \cdot \Sigma_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$$

> 💡 **경사 하강법의 일반적인 프로세스**
>
> Step 1: w_1, w_0를 임의의 값으로 설정하고 첫 비용 함수의 값 계산
> Step 2: w_1, w_0를 위 식으로 업데이트한 후 다시 비용 함수의 값 계산
> Step 3: 비용 함수의 값이 감소했으면 다시 Step 2 반복. 더 이상 감소하지 않으면 그때의 w_1, w_0를 구하고 반복 중지

실습: y = 4X + 6을 근사하기 위한 100개의 데이터 세트 생성 → 경사 하강법으로 w_1, w_0 도출

- 결과: w_1 = 4.022, w_0 = 6.162 (실제 4, 6과 유사), 예측 오류 비용 약 0.9935

코드 필사에서 이어서~

확률적 경사 하강법(Stochastic Gradient Descent)

- 일반 경사 하강법: **모든 학습 데이터**에 대해 반복적으로 비용함수 최소화를 위한 값을 업데이트 → 수행 시간이 매우 오래 걸림
- **확률적 경사 하강법**: 전체 입력 데이터가 아니라 **일부 데이터만** 이용해 w가 업데이트되는 값을 계산 → 빠른 속도 보장
- ⇒ 대용량 데이터의 경우 대부분 확률적 경사 하강법이나 **미니 배치 확률적 경사 하강법** 이용
- 실습 결과: w_1 = 4.028, w_0 = 6.156, 비용 0.9937 → 경사 하강법(0.9935)과 큰 차이 없음

코드 필사에서 이어서~

피처가 여러 개인 경우

- 피처가 M개(X_1, X_2, ..., X_{100}) → 회귀 계수도 M + 1개(1개는 w_0)
- $\hat{Y} = w_0 + w_1X_1 + w_2X_2 + \cdots + w_{100}X_{100}$
- 선형대수를 이용해 간단하게 예측값 도출: $\hat{Y}$ = np.dot(X_mat, W^T) + w_0
- w_0를 W 배열 안에 포함시키기 위해 X_mat의 맨 처음 열에 모든 값이 1인 피처 Feat 0 추가

$$\hat{Y} = X_mat \cdot W^T$$

5.4 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

LinearRegression 클래스 - Ordinary Least Squares

- 예측값과 실제 값의 RSS를 최소화해 **OLS(Ordinary Least Squares)** 추정 방식으로 구현한 클래스
- fit() 메서드로 X, y 배열을 입력 받으면 회귀 계수(Coefficients)인 W를 **coef_** 속성에 저장

```
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=False, copy_X=True, n_jobs=1)
```

구분	이름	설명
입력 파라미터	fit_intercept	불린 값, 디폴트 True. intercept(절편) 값을 계산할 것인지 말지 지정. False면 intercept가 사용되지 않고 0으로 지정
입력 파라미터	normalize	불린 값, 디폴트 False. fit_intercept가 False인 경우 무시. True면 회귀 수행 전 입력 데이터 세트 정규화
속성	coef_	fit() 수행 시 회귀 계수가 배열 형태로 저장. Shape = (Target 값 개수, 피처 개수)
속성	intercept_	intercept 값

- **다중 공선성(multi-collinearity)** 문제: OLS 기반 회귀 계수 계산은 입력 피처의 독립성에 많은 영향 → 피처 간 상관관계가 매우 높으면 분산이 매우 커져서 오류에 매우 민감해짐
  - 해결: 상관관계가 높은 피처 중 독립적인 중요한 피처만 남기고 제거 or 규제 적용

- 매우 많은 피처가 다중 공선성 문제를 가지면 PCA를 통한 차원 축소도 고려

회귀 평가 지표

- 실제 값과 회귀 예측값의 차이를 기반으로 한 지표가 중심
- 차이를 그냥 더하면 +와 -가 섞여서 오류가 상쇄됨 → 절댓값 평균, 제곱, 제곱 후 루트 등 사용

평가 지표	설명	수식
MAE	Mean Absolute Error. 실제 값과 예측값 차이를 절댓값으로 변환해 평균	$(1/n) \cdot \Sigma \lvert Y_i - \hat{Y}_i \rvert$
MSE	Mean Squared Error. 실제 값과 예측값 차이를 제곱해 평균	$(1/n) \cdot \Sigma (Y_i - \hat{Y}_i)^2$
RMSE	MSE는 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성 → MSE에 루트를 씌운 것	$\sqrt{[\ (1/n) \cdot \Sigma (Y_i - \hat{Y}_i)^2\ ]}$
R^2	분산 기반 예측 성능 평가. 실제 값의 분산 대비 예측값의 분산 비율. **1에 가까울수록 예측 정확도 높음**	예측값 Variance / 실제값 Variance

- 이 밖에 MSE·RMSE에 로그를 적용한 **MSLE(Mean Squared Log Error)**, **RMSLE(Root Mean Squared Log Error)** 도 사용
- 사이킷런은 RMSE를 제공하지 X → MSE에 제곱근을 씌워서 계산하는 함수 직접 만들어야 함

평가 방법	사이킷런 평가 지표 API	Scoring 함수 적용 값
MAE	metrics.mean_absolute_error	'neg_mean_absolute_error'
MSE	metrics.mean_squared_error	'neg_mean_squared_error'
R^2	metrics.r2_score	'r2'

💡 **scoring 파라미터의 'neg_' 접두어**

- 사이킷런의 Scoring 함수는 score 값이 **클수록 좋은** 평가 결과로 자동 평가 (특히 GridSearchCV)
- 그런데 회귀 평가 지표는 값이 커지면 오히려 나쁜 모델 → -1을 곱해 음수로 만들어 작은 오류 값이 더 큰 숫자로 인식되게 함
- ex. 10 > 1이지만 음수를 곱하면 -1 > -10
- ⇒ 'neg_mean_absolute_error' = -1 × metrics.mean_absolute_error()

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

- 보스턴 주택 가격 데이터 세트 피처

피처	설명	피처	설명
CRIM	지역별 범죄 발생률	DIS	5개 주요 고용센터까지의 가중 거리
ZN	25,000평방피트 초과 거주 지역 비율	RAD	고속도로 접근 용이도
INDUS	비상업 지역 넓이 비율	TAX	10,000달러당 재산세율
CHAS	찰스강 더미 변수(경계에 위치 1, 아니면 0)	PTRATIO	지역의 교사와 학생 수 비율

피처	설명	피처	설명
NOX	일산화질소 농도	B	지역의 흑인 거주 비율
RM	거주할 수 있는 방 개수	LSTAT	하위 계층의 비율
AGE	1940년 이전에 건축된 소유 주택 비율	MEDV	본인 소유 주택 가격(중앙값)

- 시각화(seaborn regplot) 결과
  - **RM(방 개수)**: 양 방향의 선형성(Positive Linearity)이 가장 큼 → 방 크기가 클수록 가격 증가
  - **LSTAT(하위 계층 비율)**: 음 방향의 선형성(Negative Linearity)이 가장 큼 → LSTAT이 적을수록 가격 증가
- 결과: MSE 17.306, RMSE 4.160, Variance score(R^2) 0.757
- 회귀 계수: RM이 양의 값으로 가장 큼, NOX 피처의 회귀 계수 - 값이 너무 커 보임 → 이후 최적화하면서 변화 확인
- 5 폴드 교차 검증 평균 RMSE: 5.836
  - cross_val_score(scoring="neg_mean_squared_error")로 반환된 값은 모두 음수 → -1 곱한 뒤 np.sqrt()로 RMSE 계산

코드 필사에서 이어서~

5.5 다항 회귀와 과(대)적합/과소적합 이해

다항 회귀 이해

- **다항(Polynomial) 회귀**: 회귀가 독립변수의 단항식이 아닌 2차, 3차 방정식과 같은 다항식으로 표현되는 것
  - ex. $y = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2 + w_4x_1^2 + w_5x_2^2$

> 💡 **주의! 다항 회귀는 '선형 회귀'**
>
> - 선형/비선형 회귀를 나누는 기준은 **회귀 계수**가 선형/비선형인지에 따른 것이지 독립변수의 선형/비선형 여부와는 무관
> - 위 식에서 새로운 변수 $Z = [x_1, x_2, x_1x_2, x_1^2, x_2^2]$로 두면 $y = w_0 + w_1z_1 + w_2z_2 + \cdots + w_5z_5$ → 여전히 선형 회귀

- 사이킷런은 다항 회귀 클래스를 명시적으로 제공 X → 비선형 함수를 선형 모델에 적용시키는 방법으로 구현
- **PolynomialFeatures 클래스**: degree 파라미터를 통해 입력 받은 단항식 피처를 degree에 해당하는 다항식 피처로 변환 (fit(), transform() 사용)
  - ex. $[x_1, x_2]$ → degree=2 → $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$
  - ex. degree=3 → 10개의 다항 계수로 변환
- ⇒ PolynomialFeatures로 피처 변환 후 LinearRegression으로 다항 회귀 구현

- **Pipeline** 객체를 이용하면 피처 변환과 선형 회귀 적용을 한 번에 → 코드를 더 명료하게 작성 가능

코드 필사에서 이어서~

다항 회귀를 이용한 과소적합 및 과적합 이해

- 다항식의 차수가 높아질수록 매우 복잡한 피처 간의 관계까지 모델링 가능
- but, 차수(degree)를 높일수록 학습 데이터에만 너무 맞춘 학습 → 테스트 데이터 환경에서는 오히려 예측 정확도 떨어짐 = **과적합**
- 실습: 코사인 곡선 + 잡음 데이터에 degree 1, 4, 15 적용 비교

Degree	결과	MSE
1	단순한 직선(= 단순 선형 회귀). 학습 데이터 패턴 반영 X → **과소적합**	약 0.407
4	실제 데이터 세트와 유사. 변동 잡음까지 예측하진 못했지만 테스트 데이터를 잘 예측 → **가장 뛰어난 성능**	약 0.043
15	변동 잡음 값까지 지나치게 반영. 학습 데이터만 정확히 예측, 테스트 실제 곡선과는 완전히 다른 형태 → **과적합**	182815432

- Degree 15의 회귀 계수는 Degree 1, 4와 비교할 수 없을 정도로 매우 큰 값
- ⇒ 좋은 예측 모델 = 학습 데이터의 패턴을 잘 반영하면서도 복잡하지 않은 **균형 잡힌(Balanced) 모델**

코드 필사에서 이어서~

편향-분산 트레이드오프(Bias-Variance Trade off)

- **고편향(High Bias)**: Degree 1처럼 매우 단순화된 모델, 지나치게 한 방향성으로 치우친 경향
- **고분산(High Variance)**: Degree 15처럼 학습 데이터 하나하나의 특성을 반영하면서 매우 복잡한 모델, 지나치게 높은 변동성

	Low Variance	High Variance
Low Bias	예측 결과가 실제 결과에 매우 근접 + 특정 부분에 집중 (아주 드물게 좋은 경우)	실제 결과에 비교적 근접하지만 넓은 부분에 분포
High Bias	정확한 결과에서 벗어나면서도 특정 부분에 집중	정확한 결과에서 벗어나면서 넓은 부분에 분포

일반적으로 편향과 분산은 한 쪽이 높으면 한 쪽이 낮아지는 경향

- 편향 ↑ 분산 ↓ → **과소적합**
- 분산 ↑ 편향 ↓ → **과적합**

편향을 낮추고 분산을 높이면서 전체 오류가 가장 낮아지는 **'골디락스'** 지점을 통과하면서 분산을 지속적으로 높이면 → 전체 오류 값이 오히려 증가하면서 예측 성능 다시 저하

⇒ 편향과 분산이 서로 트레이드오프를 이루면서 오류 Cost 값이 최대로 낮아지는 모델을 구축하는 것이 가장 효율적

5.6 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

규제 선형 모델의 개요

이전까지 선형 모델의 비용 함수는 RSS 최소화(실제 값과 예측값의 차이 최소화)만 고려 → 학습 데이터에 지나치게 맞추게 되고 회귀 계수가 쉽게 커짐

⇒ 비용 함수는 **학습 데이터 잔차 오류 값 최소화(RSS)** 와 **회귀 계수 크기 제어**가 서로 균형을 이뤄야 함

$$\text{비용함수목표} = Min(RSS(W) + alpha \cdot \|W\|_2^2)$$

alpha: 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터

- alpha = 0 (또는 매우 작은 값): W가 커도 alpha· ‖ W ‖ ₂가 0이 되어 비용 함수는 Min(RSS(W))
- alpha = 무한대 (또는 매우 큰 값): alpha· ‖ W ‖ ₂도 무한대가 되므로 W를 0에 가깝게 최소화해야 함

⇒ alpha ↑ → 회귀 계수 W 값을 작게 해 과적합 개선 / alpha ↓ → 회귀 계수가 커져도 상쇄 가능하므로 학습 데이터 적합 개선

- **규제(Regularization)**: 비용 함수에 alpha 값으로 페널티를 부여해 회귀 계수 값의 크기를 감소시켜 과적합을 개선하는 방식

> 💡 **L2 규제 vs L1 규제**
>
> - **L2 규제**: alpha· ‖ W ‖ ₂ → W의 **제곱**에 대해 페널티 부여 ⇒ **릿지(Ridge)** 회귀
> - **L1 규제**: alpha· ‖ W ‖ ₁ → W의 **절댓값**에 대해 페널티 부여 ⇒ **라쏘(Lasso)** 회귀
> - L1 규제를 적용하면 영향력이 크지 않은 회귀 계수 값을 0으로 변환

릿지 회귀

- 사이킷런 **Ridge** 클래스, 주요 생성 파라미터 = **alpha** (L2 규제 계수)
- 보스턴 주택 가격 alpha=10 → 5 폴드 평균 RMSE 5.524 (규제 없는 LinearRegression 5.836보다 뛰어남)
- alpha 값 변화에 따른 결과

alpha	0	0.1	1	10	100
평균 RMSE	5.836	5.796	5.659	5.524	**5.332**

- alpha 값을 계속 증가시킬수록 회귀 계수 값은 지속적으로 작아짐 (특히 NOX: -17.8 → -0.26)

but, 릿지 회귀는 회귀 계수를 **0으로 만들지는 않음**

코드 필사에서 이어서~

라쏘 회귀

- W의 절댓값에 페널티를 부여하는 L1 규제를 선형 회귀에 적용
- 비용함수 목표: $RSS(W) + alpha \cdot \|W\|_1$ 을 최소화하는 W 찾기
- L2 규제가 회귀 계수의 크기를 감소시키는 데 반해, L1 규제는 **불필요한 회귀 계수를 급격하게 감소시켜 0으로 만들고 제거** → 적절한 피처만 회귀에 포함시키는 **피처 선택**의 특성
- 사이킷런 **Lasso** 클래스, 주요 생성 파라미터 = **alpha** (L1 규제 계수)

- get_linear_reg_eval() 함수: 모델명, alpha 리스트, 피처/타깃 데이터 입력 → alpha별 폴드 평균 RMSE 출력 + 회귀 계수 DataFrame 반환

alpha	0.07	0.1	0.5	1	3
평균 RMSE	**5.618**	5.621	5.672	5.776	6.189

- alpha 0.07일 때 가장 좋음 → 릿지(5.524)보다는 약간 떨어지지만 LinearRegression(5.836)보다는 향상
- alpha 증가에 따라 일부 피처의 회귀 계수는 아예 0으로 바뀜
  - NOX: alpha 0.07부터 회귀 계수 0
  - alpha 증가 → INDUS, CHAS 등도 0
- ⇒ 회귀 계수가 0인 피처는 회귀 식에서 제외되면서 피처 선택의 효과

코드 필사에서 이어서~

엘라스틱넷 회귀

- L2 규제 + L1 규제 결합
- 비용함수 목표: RSS(W) + alpha2 · ‖ W ‖ $_2^2$ + alpha1 · ‖ W ‖ $_1$ 최소화
- 등장 배경: 라쏘는 서로 상관관계가 높은 피처들의 경우 이들 중 중요 피처만 셀렉션하고 다른 피처들은 모두 회귀 계수를 0으로 만드는 성향이 강함 → alpha 값에 따라 회귀 계수 값이 급격히 변동할 수 있음 ⇒ 이를 완화하기 위해 L2 규제를 라쏘 회귀에 추가
- 단점: L1과 L2 규제가 결합된 규제로 인해 수행시간이 상대적으로 오래 걸림
- 사이킷런 **ElasticNet** 클래스, 주요 생성 파라미터 = **alpha**, **l1_ratio**

> 💡 **ElasticNet의 alpha와 l1_ratio**
>
> - 엘라스틱넷의 규제 = a·L1 + b·L2 (a: L1 규제의 alpha 값, b: L2 규제의 alpha 값)
> - ElasticNet 클래스의 **alpha = a + b**
> - **l1_ratio = a / (a + b)**
>   - l1_ratio = 0 → a = 0 → L2 규제와 동일
>   - l1_ratio = 1 → b = 0 → L1 규제와 동일
> - cf. Ridge, Lasso 클래스의 alpha 값과는 다름!

- l1_ratio = 0.7로 고정, alpha 변화

alpha	0.07	0.1	0.5	1	3
평균 RMSE	5.546	5.530	**5.468**	5.596	6.068

- alpha 0.5일 때 가장 좋은 예측 성능, 라쏘보다는 상대적으로 0이 되는 값이 적음
- ⇒ 릿지·라쏘·엘라스틱넷 중 어떤 것이 가장 좋은지는 상황에 따라 다름. 하이퍼 파라미터를 변경하면서 최적의 예측 성능을 찾아내야 함

- but, 선형 회귀의 경우 최적의 하이퍼 파라미터를 찾아내는 것 못지않게 먼저 **데이터 분포도의 정규화와 인코딩 방법**이 매우 중요

코드 필사에서 이어서~

선형 회귀 모델을 위한 데이터 변환

- 선형 모델은 피처와 타깃값 간에 **선형의 관계**가 있다고 가정
- 피처값과 타깃값의 분포가 **정규 분포**(평균을 중심으로 종 모양) 형태를 매우 선호
    - 특히 타깃값이 특정값의 분포가 치우친 **왜곡(Skew)** 된 형태일 경우 예측 성능에 부정적인 영향 가능성 높음

⇒ 선형 회귀 모델 적용 전 데이터에 대한 스케일링/정규화 작업 수행이 일반적 (but, 무조건 성능이 향상되는 것은 X → 중요 피처들이나 타깃값의 분포도가 심하게 왜곡됐을 경우에 수행)

> 💡 **LogisticRegression 주요 하이퍼 파라미터**
>
> - **penalty**: 규제의 유형 설정. 'l2' → L2 규제, 'l1' → L1 규제 (기본은 'l2')
> - **C**: 규제 강도를 조절하는 alpha 값의 **역수** → C = 1 / alpha
>   - **C 값이 작을수록 규제 강도가 큼**

- **타깃값**은 일반적으로 **로그 변환** 적용
    - 정규 분포나 다른 정규값으로 변환하면 원본 타깃값으로 원복하기 어려움
    - 왜곡된 분포도 형태의 타깃값을 로그 변환해 성능 향상이 된 경우가 많은 사례에서 검증됨
- **np.log1p()**: 일반 log() 함수는 언더 플로우가 발생하기 쉬움 → 1 + log() 를 구현한 함수
- 보스턴 데이터 + Ridge 변환 유형별 RMSE 비교

변환 유형	alpha=0.1	alpha=1	alpha=10	alpha=100
원본 데이터	5.796	5.659	5.524	5.332
표준 정규 분포	5.834	5.810	5.643	5.424
표준 정규 분포 + 2차 다항식	8.776	6.849	5.487	**4.631**
최솟값/최댓값 정규화	5.770	5.468	5.755	7.635
최솟값/최댓값 정규화 + 2차 다항식	5.294	**4.320**	5.186	6.538
로그 변환	**4.772**	**4.676**	**4.835**	6.244

- 표준 정규 분포, 최솟값/최댓값 정규화만으로는 성능 개선 X
- 다항식 변환은 성능 개선됐지만 피처 개수가 많거나 데이터 건수가 많으면 적용에 한계
- 로그 변환은 alpha 0.1, 1, 10 모두에서 좋은 성능 향상
- ⇒ 데이터 값의 분포가 심하게 왜곡되어 있을 경우 **로그 변환**이 좋은 결과 기대 가능

코드 필사에서 이어서~

5.7 로지스틱 회귀

로지스틱 회귀 개요

- 선형 회귀 방식을 **분류**에 적용한 알고리즘 (선형 회귀 계열)
  - cf. 회귀가 선형인가 비선형인가는 독립변수가 아닌 가중치(weight) 변수가 선형인지 아닌지를 따름
- 선형 회귀와 다른 점: 선형 함수의 회귀 최적선을 찾는 것이 아니라 **시그모이드(Sigmoid) 함수** 최적선을 찾고, 이 시그모이드 함수의 반환 값을 **확률**로 간주해 확률에 따라 분류 결정

$y = 1 / (1 + e^{-x})$

- 시그모이드 함수: x 값이 +, -로 아무리 커지거나 작아져도 y 값은 항상 **0과 1 사이** 값 반환
  - x → 커지면 1에 근사, x → 작아지면 0에 근사, x = 0일 때 0.5
- ex. 종양 크기(X축)에 따라 악성 종양 여부(Y축, Yes=1 / No=0) 판단
  - 선형 회귀 라인: 0과 1을 제대로 분류 X (정확도 떨어짐)
  - S자 커브 형태의 시그모이드 함수: 좀 더 정확하게 0과 1 분류 가능

위스콘신 유방암 데이터 실습

- 선형 회귀 계열의 로지스틱 회귀는 데이터의 정규 분포도에 따라 예측 성능 영향 → StandardScaler로 표준 스케일링 후 데이터 분리
- 결과: accuracy 0.982, roc_auc 0.979

> 💡 **LogisticRegression 주요 하이퍼 파라미터**
>
> - **penalty**: 규제의 유형 설정. 'l2' → L2 규제, 'l1' → L1 규제 (기본은 'l2')
> - **C**: 규제 강도를 조절하는 alpha 값의 **역수** → $C = \frac{1}{alpha}$
>   - **C 값이 작을수록 규제 강도가 큼**

- GridSearchCV 결과: 최적 하이퍼 파라미터 {'C': 0.1, 'penalty': 'l2'}, 최적 평균 정확도 0.979
- 로지스틱 회귀는 가볍고 빠르지만 이진 분류 예측 성능도 뛰어남 → 이진 분류의 기본 모델로 많이 사용
- 희소한 데이터 세트 분류에도 뛰어난 성능 → 텍스트 분류에서도 자주 사용

코드 필사에서 이어서~

5.8 회귀 트리

회귀 트리 개요

- 선형 회귀: 회귀 계수를 선형으로 결합하는 회귀 함수를 구해 독립변수를 입력해 결괏값 예측
- 비선형 회귀: 회귀 계수의 결합이 비선형
- ⇒ 머신러닝 기반의 회귀 = 회귀 계수를 기반으로 하는 최적 회귀 함수를 도출하는 것이 주요 목표
- **회귀 트리**: 회귀 함수를 기반으로 하지 않고 결정 트리와 같이 **트리를 기반**으로 하는 회귀 방식
- 분류 트리와 크게 다르지 않음. 다만 **리프 노드에서 예측 결정 값을 만드는 과정**에 차이

	분류 트리	회귀 트리
리프 노드 결정 값	특정 클래스 레이블 결정	리프 노드에 속한 데이터 값의 **평균값**을 구해 회귀 예측값 계산

- 동작 과정: X 피처를 결정 트리 기반으로 분할 (Split 0 → Split 1, 2 → Split 3 ...) → 트리 규칙으로 변환 → 리프 노드 생성 기준에 부합하는 분할이 완료되면 리프 노드에 소속된 데이터 값의 평균값을 구해 최종적으로 리프 노드에 결정 값으로 할당

CART 알고리즘과 사이킷런 회귀 트리

- 결정 트리, 랜덤 포레스트, GBM, XGBoost, LightGBM 등 4장의 모든 트리 기반 알고리즘은 분류뿐 아니라 회귀도 가능
    - 트리 생성이 **CART(Classification And Regression Trees)** 알고리즘에 기반하기 때문

알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

- 보스턴 주택 가격 5 교차 검증 평균 RMSE

모델	평균 RMSE
DecisionTreeRegressor	5.958
RandomForestRegressor	4.423
GradientBoostingRegressor	4.195
XGBRegressor	**4.109**
LGBMRegressor	4.598

- 회귀 트리 Regressor 클래스는 coef_ 속성 X → 대신 **feature_importances_** 를 이용해 피처별 중요도 확인 (보스턴: RM, LSTAT 순으로 중요)
- 회귀 트리 Regressor의 하이퍼 파라미터는 분류 트리 Classifier의 하이퍼 파라미터와 거의 동일

코드 필사에서 이어서~

선형 회귀 vs 회귀 트리 예측선 비교

- RM 피처만 이용해 LinearRegression과 DecisionTreeRegressor(max_depth = 2, 7) 비교
    - 선형 회귀: **직선**으로 예측 회귀선 표현
    - 회귀 트리: 분할되는 데이터 지점에 따라 브랜치를 만들면서 **계단 형태**로 회귀선 만듦
    - max_depth=7: 학습 데이터 세트의 이상치(outlier) 데이터도 학습하면서 복잡한 계단 형태 → **과적합**되기 쉬운 모델

코드 필사에서 이어서~

5.11 정리

- **선형 회귀**: 실제값과 예측값의 차이인 오류를 최소로 줄일 수 있는 선형 함수를 찾아서, 이 선형 함수에 독립변수(피처)를 입력해 종속변수(타깃값, 예측값)를 예측
    - 실제값과 예측값 차이의 제곱을 회귀 계수 W를 변수로 하는 **비용 함수**로 만들고, 이 비용 함수가 최소화되는 W를 찾음
    - 비용 함수를 최소화하는 방법 = **경사 하강법** (머신러닝 전반에 걸쳐 매우 중요한 개념)
- 실제값과 예측값 차이를 최소화하는 것에만 초점을 맞춘 단순 선형 회귀는 과적합 문제 수반 가능성 높음 → **규제(Regularization)** 도입
    - 릿지(L2), 라쏘(L1), 엘라스틱넷(L1 + L2)
- **로지스틱 회귀**: 선형 회귀를 분류에 적용한 대표 모델. 선형 함수 대신 최적의 시그모이드 함수를 도출하고, 독립변수를 시그모이드 함수에 입력해 반환된 결과를 확률값으로 변환해 예측 레이블 결정. 이진 분류나 희소 행렬로 표현되는 텍스트 기반 분류에서 높은 성능
- **회귀 트리**: 분류 트리와 크게 다르지 않으나 리프 노드에 속한 데이터 값의 평균값을 구해 회귀 예측값 계산. 결정 트리, 랜덤 포레스트, GBM, XGBoost, LightGBM 모두 회귀 가능
- 선형 회귀는 데이터 값의 분포도와 인코딩 방법에 많은 영향
    - 정규 분포와 같은 종 모양 형태 선호, 특히 타깃값의 분포도가 왜곡(Skew)되지 않아야 함 → 왜곡 시 일반적으로 **로그 변환** 적용
    - 카테고리형 데이터는 레이블 인코딩보다는 **원-핫 인코딩**으로 변환해야 함 (회귀 트리는 인코딩 방식에 크게 영향 X)
- 스태킹 모델을 회귀에 적용해 훌륭한 예측 성능 도출